

# **Software Requirements Specifications**

## **AI-Powered Automated SRS Generator**

**Internal Advisor:** Dr. Fahad Maqbool

**Project Manager:** Dr. Saad Razzaq

### **Project Team**

M Talal Ahmed BSCS51F22S074

M Tayyab BSCS51F22S070

M Ibrahim BSCS51F22S078

# Table of Contents

|     |                                       |   |
|-----|---------------------------------------|---|
| 1.  | Introduction .....                    | 3 |
| 1.1 | Purpose of Document .....             | 3 |
| 1.2 | Project Overview .....                | 3 |
| 1.3 | Scope .....                           | 3 |
| 2.  | Overall System Description .....      | 4 |
| 2.1 | User Characteristics .....            | 4 |
| 2.2 | Operating Environment .....           | 4 |
| 2.3 | System Constraints .....              | 5 |
| 3.  | External Interface Requirements ..... | 5 |
| 3.1 | Hardware Interfaces .....             | 5 |
| 3.2 | Software Interfaces .....             | 5 |
| 4.  | Functional Requirements .....         | 6 |
| 5.  | Non-functional Requirements .....     | 7 |
| 5.1 | Performance Requirements .....        | 7 |
| 5.2 | Safety Requirements .....             | 8 |
| 5.3 | Security Requirements .....           | 8 |
| 5.4 | User Documentation .....              | 8 |
| 6.  | References .....                      | 8 |
| 7.  | Appendix .....                        | 9 |
| 7.1 | Use Cases .....                       | 9 |

# 1. Introduction

The Software Requirements Specification (SRS) document defines the current architecture and behavior of the AI-Driven SRS Generator. The system is now implemented as a split application with a Next.js 16 frontend, a FastAPI and LangGraph backend, a PostgreSQL persistence layer, and a ChromaDB retrieval store. It converts a user's project idea into a structured SRS draft through guided chat, clarification questions, live section drafting, Mermaid diagram generation, and DOCX export.

## 1.1 Purpose of Document

This document describes the current product architecture for developers, testers, maintainers, and reviewers. It reflects the live implementation rather than the original target design. The document covers the system context, major components, user-facing capabilities, interfaces, and core functional and non-functional requirements.

## 1.2 Project Overview

The AI-Driven SRS Generator is a web-based application for turning plain-language software ideas into an IEEE-style SRS draft. Users interact with a chat workspace in the frontend. The frontend manages authentication, chat sessions, progress polling, and file export. The backend runs a LangGraph workflow that retrieves reference material, elicits missing details, drafts SRS sections, generates Mermaid diagrams when requested, and returns the assembled document.

The current architecture includes:

- Next.js 16 frontend with React 19 and Prisma.
- FastAPI backend serving the SRS generation API.
- LangGraph StateGraph for orchestration and checkpointed runs.
- PostgreSQL for app data and LangGraph checkpoints.
- ChromaDB for seeded regulatory and standards context.
- OpenRouter for model access, including a main generation model and a smaller guardrail model.
- Markdown rendering in the UI and DOCX export from the backend.

Key benefits of the current system include:

- Guided chat-based requirement elicitation.
- Live section previews and progress indicators.
- Persistent chat and session history.
- Diagram generation with Mermaid.
- Export of the final document as DOCX.

## 1.3 Scope

In scope (what the system will do):

- User authentication with email and password.
- Chat-based requirement gathering and clarification.
- SRS generation through a LangGraph workflow.
- Live preview of draft sections and final Markdown document.
- Mermaid diagram generation and correction.
- DOCX export of the generated SRS.
- Session persistence, chat history, and resumable runs.

Out of scope (what the system will not do):

- PDF export.
- Wireframe generation.
- Source code generation.
- Real-time collaborative editing by multiple users.
- Project management, scheduling, or task tracking.
- Permanent storage of user content by default without user action.

## 2. Overall System Description

The system operates as a two-tier web application. The browser hosts the Next.js frontend, and the backend is a separate FastAPI service. The frontend communicates with the backend through REST endpoints and Server-Sent Events for streaming updates. PostgreSQL stores users, chats, messages, runs, and timing metadata. The backend uses ChromaDB and OpenRouter to generate responses and to ground the draft in standards and guidance.

### 2.1 User Characteristics

The system is designed to accommodate multiple user classes with varying technical expertise:

#### 2.1.1 Primary User Classes:

##### 1. Non-Technical Users/Entrepreneurs

- Technical Background: Limited or no programming knowledge
- Documentation Experience: None to low; unfamiliar with SRS structure
- Frequency of Use: Occasional, typically for single projects
- Goals: Transform ideas into structured, IEEE-aligned SRS with minimal jargon and guided prompts

##### 2. Requirements Engineers

- Technical Background: Strong understanding of requirements engineering and SDLC
- Documentation Experience: High; expert in requirements analysis and specification
- Frequency of Use: Regular, across multiple projects and iterations
- Goals: Generate high-quality drafts quickly, enforce consistency/traceability, and prepare documents for stakeholder review and baselining

#### 2.1.2 Secondary User Classes:

##### 3. Academic Students

- Technical Background: Basic to intermediate programming knowledge
- Documentation Experience: Limited or no prior SRS writing experience
- Frequency of Use: Periodic (primarily during project initiation phases)
- Goals: Create IEEE-compliant SRS documents for academic project submissions

##### 4. Software Developers

- Technical Background: Strong programming and software engineering knowledge
- Documentation Experience: Moderate; understands requirements but seeks efficiency
- Frequency of Use: Regular, for multiple projects
- Goals: Accelerate documentation process while maintaining quality standards

##### 5. Project Managers

- Technical Background: Broad understanding of software delivery and stakeholder needs
- Documentation Experience: Moderate; focuses on scope, timelines, and stakeholder alignment
- Frequency of Use: Regular during planning and review phases
- Goals: Produce review-ready drafts for planning, communication, and approvals

### 2.2 Operating Environment

#### 2.2.1 Hardware Platform:

- **Client Side:** Any device capable of running modern web browsers (desktop computers, laptops, tablets)
- **Server Side:**
  - Python 3.12+ runtime for the backend.
  - Node.js 20+ runtime for the frontend.
  - PostgreSQL for persistence.
  - ChromaDB for retrieval context.
  - OpenRouter access for model calls.

### 2.2.2 *Operating Systems*

- **Client:** Any operating system with modern browsers
- **Server:** Modern Windows or Linux distributions

### 2.2.3 *Network Requirements*

- Internet connectivity for accessing the web application
- HTTPS support for secure data transmission
- Minimum bandwidth: 1 Mbps for acceptable performance

### 2.2.4 *Co-existing Software*

- The system will generate documents compatible with Microsoft Word, Google Docs

## 2.3 System Constraints

### 2.3.1 *Software Constraints*

- The workflow is centered on chat and SSE streaming.
- Document generation must complete within 5 minutes to maintain user engagement
- The backend graph compiles at startup and runs with checkpointed state.
- Mermaid diagrams are generated and validated by the backend

### 2.3.2 *Operational Constraints*

- The system depends on external model availability
- Session and chat data are stored in the shared PostgreSQL instance
- The DOCX export path depends on backend document rendering logic

### 2.3.3 *Security Constraints*

- Authentication uses signed session cookies in the frontend
- User content is handled as application data and should not be persisted beyond the intended session lifecycle without user action

## 3. External Interface Requirements

### 3.1 Hardware Interfaces

The system has no specialized hardware requirements. It uses standard keyboard, mouse, and display devices through the browser. On the server, CPU resources are required for application logic and model orchestration.

### 3.2 Software Interfaces

#### 3.2.1 *Web Browser (Client-Side)*

**Name:** Modern web browsers (Chrome, Firefox, Edge, Safari)

**Version:** Latest stable versions (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+)

**Purpose:** Render user interface and handle client-side interactions

**Interface Type:** HTTP/HTTPS RESTful API communication

**Data Exchange:** JSON for API requests/responses, binary for file downloads

#### 3.2.2 *Frontend Application*

**Name:** Next.js 16 frontend.

**Purpose:** Provide authentication, chat workspace, preview rendering, and export actions.

**Interface Type:** REST calls to local API routes and backend proxying.

### 3.2.3 *Backend Service*

**Name:** FastAPI backend.

**Purpose:** Run the SRS generation workflow and provide document and state endpoints.

**Interface Type:** JSON REST endpoints and SSE streams.

### 3.2.4 *Workflow Engine*

**Name:** LangGraph StateGraph.

**Purpose:** Coordinate retrieval, elicitation, drafting, QA, and diagram generation.

**Interface Type:** In-process Python API.

### 3.2.5 *Retrieval Store*

**Name:** ChromaDB.

**Purpose:** Store seeded standards and reference text for retrieval-augmented generation.

**Interface Type:** Python library API.

### 3.2.6 *Database*

**Name:** PostgreSQL.

**Purpose:** Store users, chats, messages, runs, and checkpoint data.

**Interface Type:** Prisma on the frontend side and psycopg-based access on the backend side.

### 3.2.7 *Model Provider*

**Name:** OpenRouter.

**Purpose:** Provide access to the generation model and the guardrail classifier model.

**Interface Type:** HTTP API.

### 3.2.8 *Document Export*

**Name:** DOCX export pipeline.

**Purpose:** Convert generated Markdown to a Word document.

**Interface Type:** Backend file generation and browser download.

## 4. Functional Requirements

### 4.1.1 *FR-1: Authentication*

The system shall allow users to sign up, log in, and log out using the Next.js frontend and signed session cookies.

### 4.1.2 *FR-2: Chat Session Management*

The system shall allow a user to create, open, resume, and list chat-based SRS sessions.

### 4.1.3 *FR-3: User Input Collection*

The system shall accept natural-language project descriptions and follow-up answers in a chat interface.

### 4.1.4 *FR-4: Guardrail Classification*

The system shall classify user messages so that unrelated or unsafe inputs can be redirected away from the SRS workflow.

#### 4.1.5 *FR-5: Requirement Elicitation*

The system shall ask clarifying questions when the submitted project description is incomplete or ambiguous.

#### 4.1.6 *FR-6: AI-Powered Draft Generation*

The system shall generate an SRS draft using the backend workflow, retrieval context, and model output.

#### 4.1.7 *FR-7: Section Drafting*

The system shall draft the main SRS sections, including introduction, overall description, interface requirements, functional requirements, and non-functional requirements.

#### 4.1.8 *FR-8: Live Preview*

The system shall show live section previews and the assembled draft in the frontend workspace while generation is in progress.

#### 4.1.9 *FR-9: Section Revision*

The system shall allow targeted revision of selected sections without forcing the user to restart the full document flow.

#### 4.1.10 *FR-10: Diagram Generation*

The system shall generate Mermaid diagrams when requested or when the workflow determines they are needed.

#### 4.1.11 *FR-11: Document Export*

The system shall export the completed SRS as a DOCX file that can be downloaded by the user.

#### 4.1.12 *FR-12: Session Persistence*

The system shall persist chat state, draft state, and run status so users can resume work later.

#### 4.1.13 *FR-13: Progress Feedback*

The system shall display progress updates during generation, including stage changes and running status.

#### 4.1.14 *FR-14: Error Handling*

The system shall provide readable error messages when generation, retrieval, or export fails.

#### 4.1.15 *FR-15: Searchable History*

The system shall keep a record of chats, messages, and run metadata for later review.

## 5. Non-functional Requirements

### 5.1 Performance Requirements

#### 5.1.1 *NFR-P1: Response Time*

The system should stream progress quickly after a request is submitted and complete typical SRS generation within a practical user-facing time window.

#### 5.1.2 *NFR-P2: Concurrent Users*

The system should support multiple simultaneous chat sessions without blocking the UI.

#### 5.1.3 *NFR-P3: Export Speed*

DOCX export should complete within a short user-visible delay for typical documents.

#### 5.1.4 *NFR-P4: Availability*

The system should remain available during normal operational use and recover cleanly after backend restarts.

## 5.2 Safety Requirements

#### 5.2.1 *NFR-S1: Data Loss Prevention*

The system should preserve chat and run state during normal interruptions so generation can be resumed.

#### 5.2.2 *NFR-S2: Graceful Degradation*

If the model provider or retrieval store is unavailable, the system should fail with a clear message rather than corrupting saved data.

#### 5.2.3 *NFR-S3: Input Sanitization*

The system should sanitize user input and generated output before rendering or storage.

## 5.3 Security Requirements

#### 5.3.1 *NFR-SE1: Transport Security*

Client and server communication should use HTTPS in production.

#### 5.3.2 *NFR-SE2: Authentication Security*

Passwords should be hashed, and session cookies should be protected with secure browser flags.

#### 5.3.3 *NFR-SE3: Access Control*

Users should only access their own chats and generated documents.

#### 5.3.4 *NFR-SE4: Privacy*

User content should be retained only as long as required for the active application flow or explicit storage.

## 5.4 User Documentation

#### 5.4.1 *NFR-UD1: Help Content*

The system should provide clear in-app guidance for creating chats, answering questions, reviewing drafts, and exporting documents.

#### 5.4.2 *NFR-UD2: Usability*

The interface should be simple enough for non-technical users to complete the generation flow without training.

## 6. References

IEEE 830-1998 - IEEE Recommended Practice for Software Requirements Specifications.

FastAPI documentation.

LangGraph documentation.

Next.js documentation.

Prisma documentation.

PostgreSQL documentation.



ChromaDB documentation.

OpenRouter documentation.

## 7. Appendix

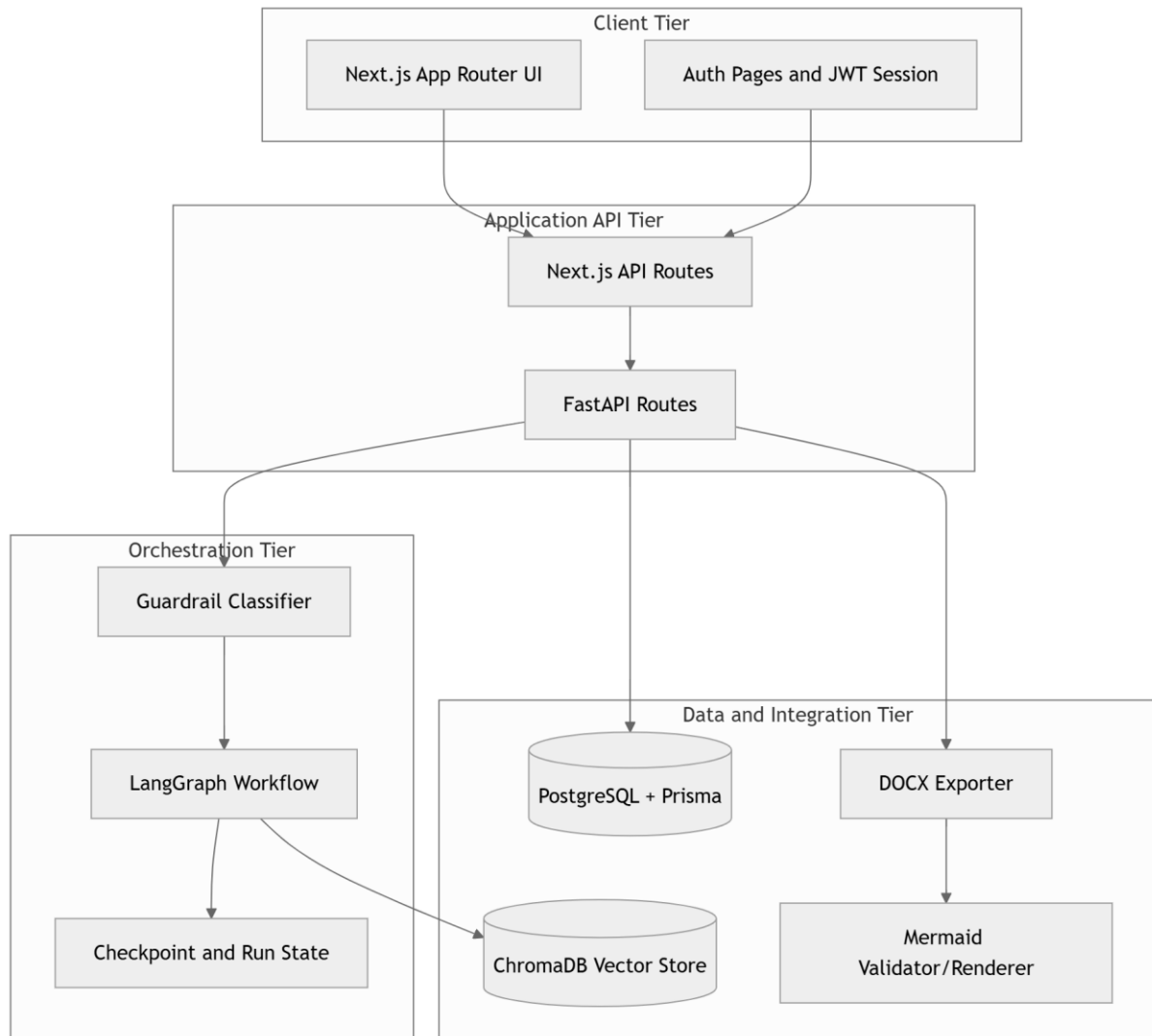


Figure 1. System Architecture Diagram showing the key components of the application

### 7.1 Use Cases

| Use Case ID | Use Case Name        | Primary Actor | Brief Description                                                                                  |
|-------------|----------------------|---------------|----------------------------------------------------------------------------------------------------|
| UC-1        | Create or Sign In    | User          | The user creates an account or signs in to access the workspace.                                   |
| UC-2        | Start a New Chat     | User, System  | The user starts a new elicitation session and the system creates a persisted chat record.          |
| UC-3        | Provide Requirements | User          | The user describes the project and answers clarification questions until the workflow can proceed. |

---

|             |                           |              |                                                                                                             |
|-------------|---------------------------|--------------|-------------------------------------------------------------------------------------------------------------|
| <b>UC-4</b> | Generate and Review Draft | User, System | The system drafts the SRS sections, streams progress, and shows live preview cards in the workspace.        |
| <b>UC-5</b> | Export Final Document     | User, System | The user downloads the final document as DOCX after reviewing the draft.                                    |
| <b>UC-6</b> | Revise a Section          | User, System | The user requests a targeted revision of one section and the system updates only that portion of the draft. |